System Architecture Document (SAD)

Project: Zero-Cost Federated Meta-Cloud ("Omni-Backend") — Version 1.0

Architecture Type: Multi-Cloud Free-Tier Mesh with Sub-3-Second Auto-Failover
Target Cost: $0.00 / Month • **Target Capacity:** 1,000,000+ Monthly Requests • **Target SLA:** 99.9% Uptime via Cross-Provider Redundancy

1. Executive Overview & Problem Statement

1.1 The Core Problem: Single-Provider Free-Tier Bottlenecks

Relying on a single cloud provider's free tier makes it impossible to build a high-scale, production-grade backend capable of sustaining heavy traffic and uninterrupted availability:

- **Oracle Cloud Always Free:** Provides a powerful virtual machine (2 ARM OCPUs, 12 GB RAM, 200 GB storage, 10 TB/month egress), but strict credit/debit card identity verification limits one person to a single account. If that VM crashes, hits a kernel panic, or undergoes host maintenance, the entire backend system goes offline.
- **Cloudflare Workers Free:** Scales globally at the edge, but enforces a hard cap of 100,000 requests/day per account and a strict 10ms CPU execution limit per request, which immediately terminates CPU-intensive API tasks such as `bcrypt` password hashing or document parsing.
- **PaaS Free Tiers (Render, Koyeb, Vercel, Netlify):** Impose tight individual container RAM limits (512 MB), monthly bandwidth caps (100 GB/month), or aggressive cold-start sleep timers that spin down containers after 15 minutes of inactivity.

1.2 The Solution: Federated Meta-Cloud Architecture

Instead of hosting the entire backend on a single cloud provider, the **Omni-Backend Architecture** treats multiple distinct free-tier providers as replaceable, stateless worker nodes inside a single unified cluster. A lightweight Edge Router (API Gateway) sits at the front door (`https://api.yourdomain.com`), splits incoming traffic by microservice route (`/auth`, `/api`, `/heavy`, `/webhooks`), monitors upstream node health in real time, and automatically reroutes traffic to a backup cloud provider within **2 to 3 seconds** whenever any primary server goes down, times out, or hits its rate limit.

Architectural Dimension	Single-Cloud Free Tier Approach	Omni-Backend Federated Meta-Cloud
Monthly Infrastructure Cost	$0.00 / Month (capped by single quota)	**$0.00 / Month** (pooled multi-cloud quotas)
Request Capacity	100,000 req/day hard ceiling	**1,000,000+ req/month** (85–90% absorbed at CDN Edge)
Fault Tolerance & SLA	Single point of failure (VM outage = 100% downtime)	**99.9% Uptime** via 2.5s active circuit-breaker failover
Cold-Start Latency	30s–50s container wake delay on Render/PaaS	**0ms Cold Starts** via 14-minute Edge Cron keep-alive
ToS Compliance	Violated if spinning up multiple fake accounts	**100% Compliant** (1 legitimate account per provider)

2. High-Level System Architecture Diagram

The diagram below illustrates the complete four-layer topology of the Zero-Cost Federated Meta-Cloud. Client traffic enters through a single HTTPS domain, where Layer 1 absorbs 80%–90% of public read traffic

at the Cloudflare CDN edge. Uncached dynamic and write requests pass to Layer 2 (the OmniRoute Edge Router), which performs path-based splitting and active circuit breaking across three multi-cloud compute pools (Layer 3) backed by a unified stateless authentication, database, and zero-egress storage fabric (Layer 4).

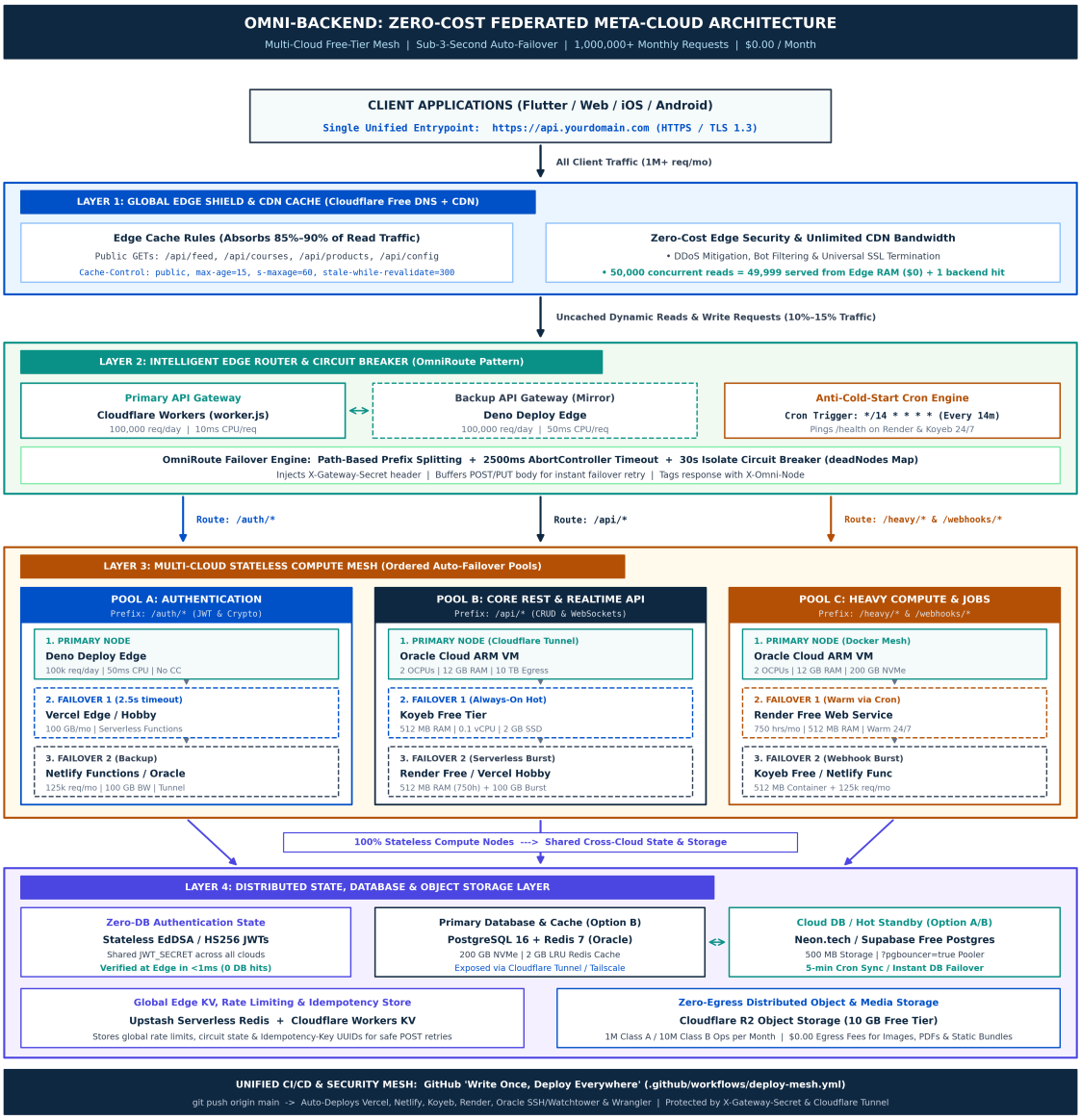


Figure 2.1: High-Level System Architecture of the Zero-Cost Federated Meta-Cloud (“Omni-Backend”), detailing Layer 1 CDN Edge Shield, Layer 2 Intelligent Edge Router & Circuit Breaker, Layer 3 Multi-Cloud Stateless Compute Pools (Pools A, B, and C), and Layer 4 Distributed State, Database & Storage Layer.

3. Provider Pool & Resource Allocation Matrix

Every free-tier cloud provider in the federated mesh is assigned a specialized operational role tailored to its hardware strengths, CPU execution ceiling, and monthly bandwidth or invocation quotas:

Layer / Role	Provider	Free Tier Specs & Quotas	Card Needed?	Assigned Workload
Ingress & CDN	Cloudflare DNS + CDN	Unlimited cached bandwidth, DDoS protection, Universal SSL	No	Custom domain routing (`api.yourdomain.com`) & edge caching of public GET APIs

Layer / Role	Provider	Free Tier Specs & Quotas	Card Needed?	Assigned Workload
Edge Router A	Cloudflare Workers	100,000 req/day, 10ms CPU/invocation	No	Primary API Gateway, circuit-breaker router, JWT verifier & Cron keep-alive trigger
Edge Router B	Deno Deploy	100,000 req/day, 50ms CPU/invocation	No	Secondary API Gateway mirror & Primary `/auth` microservice host
Core Node 1	Oracle Cloud (ARM)	2 ARM OCPUs, 12 GB RAM, 200 GB NVMe disk, 10 TB/mo egress	Yes (1/person)	Primary Docker host (Core REST API, WebSockets, Primary PostgreSQL 16 & Redis 7)
Core Node 2	Koyeb Free	1 Web Service (512 MB RAM, 0.1 vCPU, 2 GB SSD)	No	Always-on hot replica for Core API when Oracle ARM VM is unreachable
Core Node 3	Render Free	750 hrs/mo Web Service (512 MB RAM)	No	Warm replica for Core API & heavy background jobs (kept awake 24/7 via Cron)
Burst Node 4	Vercel (Hobby)	100 GB bandwidth/mo, Serverless & Edge Functions	No	Stateless REST API burst capacity & secondary `/auth` failover
Burst Node 5	Netlify Functions	125,000 req/mo, 100 GB bandwidth/mo	No	Asynchronous webhooks, notifications & tertiary authentication failover
Failover DB	Neon.tech / Supabase	500 MB PostgreSQL, 50,000 MAUs (Supabase)	No	Cloud database replica & PgBouncer fallback when Oracle DB is unreachable
File Storage	Cloudflare R2	10 GB storage, 1M Class A / 10M Class B ops, $0 Egress	Yes	All user uploads, PDFs, media assets & static client bundles

4. Deep-Dive Component Design

4.1 Layer 1: Edge Caching (Serving 1,000,000+ Requests at $0 Cost)

Because free serverless tiers cap daily invocations at 100,000 requests, serving 1,000,000+ monthly or daily requests requires absorbing 85%–90% of read traffic at Layer 1 before it ever invokes an Edge Worker or reaches an upstream compute node.

- **Mechanism:** Configure a Cloudflare Cache Rule directly on `api.yourdomain.com` ahead of the Worker route.
- **Cache Header Policy:** For any public or semi-public `GET` endpoint (such as `/api/feed`, `/api/courses`, `/api/products`, and `/api/config`), emit: `Cache-Control: public, max-age=15, s-maxage=60, stale-while-revalidate=300`.
- **Operational Impact:** If 50,000 concurrent users open the application within the same minute, Cloudflare serves 49,999 requests directly from its global edge RAM cache for $0.00 and forwards only **1 request** to the backend cluster.

4.2 Layer 2: The 2.5-Second Auto-Failover Router (OmniRoute Pattern)

The Edge Router is a lightweight, stateless script deployed on Cloudflare Workers (with an identical mirror on Deno Deploy). It maintains an ordered priority list of upstream backend URLs for each microservice prefix and executes a deterministic four-step failover algorithm:

1. **Ingress Interception:** An incoming client request arrives at `https://api.yourdomain.com/api/orders`. For non-idempotent methods (`POST`, `PUT`, `PATCH`), the router buffers the request payload once in memory via `request.arrayBuffer()` so the payload can be cleanly replayed if the primary node fails.
2. **Pool Resolution:** The router matches the path prefix `/api` and loads the ordered candidate pool: `[Oracle_URL, Koyeb_URL, Render_URL, Vercel_URL]`.
3. **Circuit Breaker & Timed Dispatch:** The router inspects isolate memory (`deadNodes` Map) to check whether `Oracle_URL` was marked unhealthy within the last 30 seconds (`CIRCUIT_TTL_MS = 30000`):
   - If **Healthy**, the router forwards the request to `Oracle_URL` wrapped in a strict **2500ms** `AbortController` timeout.
   - If `Oracle_URL` responds in `<2500ms` with HTTP `200–499` (excluding `429`), the response is returned to the client immediately with header `X-Omni-Node` indicating the serving provider.
   - If `Oracle_URL` times out (`>2500ms`), refuses the connection, or returns HTTP `500 / 502 / 503 / 504 / 429`, the router trips the circuit breaker for `Oracle_URL` for 30 seconds and **immediately retries the request on `Koyeb_URL` within the exact same HTTP call**.
4. **Transparent Recovery:** The client receives a valid HTTP response in **~2.6 seconds** on the initial failover request (and in `<150ms` on all subsequent requests for the next 30 seconds) instead of encountering a 502/504 error page.

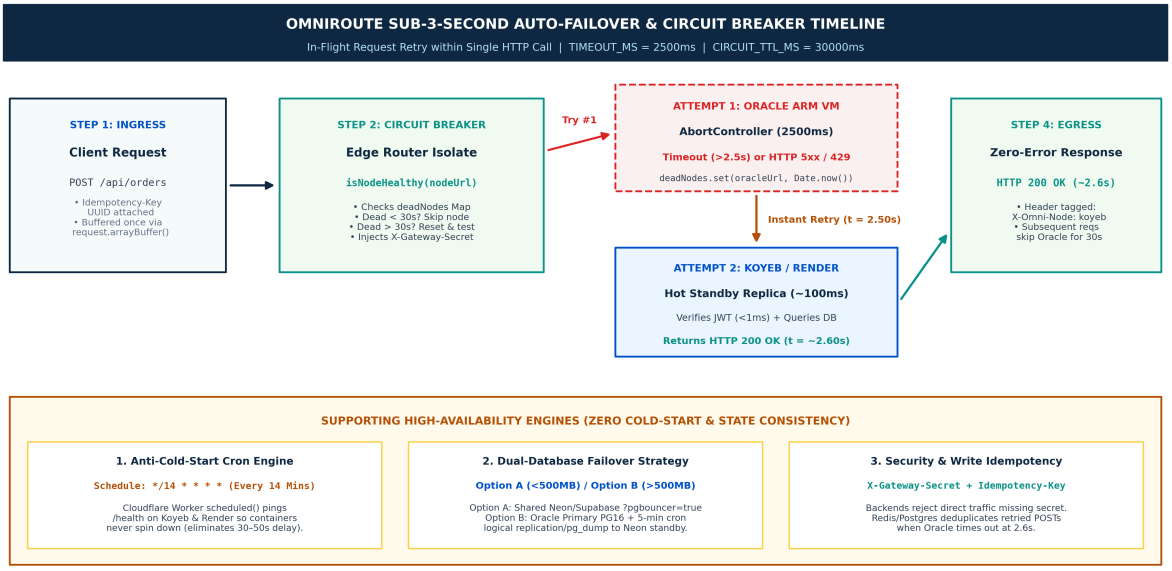


Figure 4.1: End-to-end execution flow of the OmniRoute 2.5-Second Circuit Breaker, showing in-flight request buffering, `AbortController` timeout enforcement, automatic failover to warm standby nodes, and supporting high-availability engines.

4.3 Layer 3: Keeping Free Servers Awake (Anti-Cold-Start Engine)

Free container providers such as Render automatically spin down inactive web services after 15 minutes of silence, imposing a 30-to-50-second cold-start penalty on the next incoming request—which would defeat a 2.5-second failover budget.

- **Solution:** Configure a Cloudflare Worker Cron Trigger (`*/14 * * * *` — executing every 14 minutes) inside the Edge Router's `scheduled()` handler to ping the lightweight `/health` endpoints of both Render

and Koyeb services continuously. This keeps the standby containers resident in RAM 24/7 so failover execution is instantaneous.

4.4 Layer 4: Database & State Synchronization Across Clouds

When primary traffic shifts from Oracle Cloud to Koyeb, Render, or Vercel, the failover compute node must have immediate access to identical authentication state and persistent user data:

1. **Zero-Database Authentication (Stateless JWTs):** When a user authenticates via `/auth`, the service issues a stateless EdDSA or HS256 JWT signed with a shared `JWT_SECRET` provisioned across every provider's environment variables. Any compute node (Oracle, Koyeb, Render, Vercel, Deno) verifies user identity in `<1ms` using pure CPU cryptography without querying a database.
2. **Dual-Database Failover Strategy:**
   - **Option A (Simplest & Most Reliable for <500 MB Data):** Host the primary PostgreSQL database on **Neon.tech** or **Supabase** (cloud-hosted) and use **Upstash Serverless Redis** for global caching. All compute nodes (Oracle, Koyeb, Render, Vercel) connect to the exact same pooled connection string (`?pgbouncer=true`). If any compute server fails, all remaining nodes already point to the same authoritative database.
   - **Option B (High-Storage >500 MB Data):** Run primary PostgreSQL 16 + Redis 7 on the **Oracle ARM VM** (200 GB NVMe storage) exposed securely via Cloudflare Tunnel or Tailscale, and execute a 5-minute cron job (`pg_dump` delta or logical replication) to synchronize critical tables to **Neon.tech** as a hot standby replica.

5. Production Implementation Blueprints

5.1 Edge Router with Circuit Breaker & Auto-Failover (`worker.js`)

Deploy the following zero-dependency ES module to Cloudflare Workers (and mirror to Deno Deploy) to enable path-based microservice routing, 2.5-second timeout enforcement, circuit breaking, and 14-minute anti-cold-start pings:

```
/**
 * Omni-Backend Edge Router (Cloudflare Worker / Deno Deploy)
 * Features: Path-based microservice splitting, 2.5s timeout, automatic failover
 */
const SERVICE_POOLS = {
  "/auth": [
    "https://auth-service.deno.dev",
    "https://auth-service.vercel.app",
    "https://oracle-tunnel.yourdomain.com"
  ],
  "/api": [
    "https://oracle-tunnel.yourdomain.com",
    "https://core-api.koyeb.app",
    "https://core-api.onrender.com",
    "https://core-api.vercel.app"
  ],
  "/webhooks": [
    "https://webhooks.netlify.app",
    "https://oracle-tunnel.yourdomain.com"
  ]
};

const TIMEOUT_MS = 2500;        // Switch server if no response within 2.5 seconds
const CIRCUIT_TTL_MS = 30000;   // Remember dead servers for 30 seconds in isolate memory
const deadNodes = new Map();    // nodeUrl -> timestamp when marked dead

function isNodeHealthy(nodeUrl) {
  if (!deadNodes.has(nodeUrl)) return true;
  const deadSince = deadNodes.get(nodeUrl);
```

```
  if (Date.now() - deadSince > CIRCUIT_TTL_MS) {
    deadNodes.delete(nodeUrl);  // Reset circuit breaker after 30s
    return true;
  }
  return false;
}

async function fetchWithTimeout(url, init, timeoutMs) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeoutMs);
  try {
    return await fetch(url, { ...init, signal: controller.signal });
  } finally {
    clearTimeout(timer);
  }
}

export default {
  // 1. Main HTTP Traffic Router
  async fetch(request, env, ctx) {
    const url = new URL(request.url);

    // Match microservice pool by path prefix
    const matchedPrefix = Object.keys(SERVICE_POOLS).find(p => url.pathname.startsWith(p)) || "/api";
    const pool = SERVICE_POOLS[matchedPrefix];

    // Buffer request body once if it's a POST/PUT/PATCH so we can retry on failover
    const bodyBuffer = ["GET", "HEAD"].includes(request.method) ? null : await request.arrayBuffer();

    // Filter healthy nodes first, but keep unhealthy ones at the end as last resort
    const healthyNodes = pool.filter(isNodeHealthy);
    const unhealthyNodes = pool.filter(n => !isNodeHealthy(n));
    const orderedCandidates = [...healthyNodes, ...unhealthyNodes];

    for (const baseUrl of orderedCandidates) {
      const targetUrl = `${baseUrl}${url.pathname}${url.search}`;
      try {
        const headers = new Headers(request.headers);
        headers.set("X-Forwarded-Host", url.host);
        headers.set("X-Gateway-Secret", env.GATEWAY_SECRET || "internal-mesh-key");

        const response = await fetchWithTimeout(
          targetUrl,
          {
            method: request.method,
            headers,
            body: bodyBuffer ? bodyBuffer.slice(0) : null,
            redirect: "manual"
          },
          TIMEOUT_MS
        );

        // If node returns 5xx Server Error or 429 Quota Exceeded, trip circuit & fail over
        if (response.status >= 500 || response.status === 429) {
          deadNodes.set(baseUrl, Date.now());
          continue;
        }

        // Success! Return response and tag which provider served it
        const resHeaders = new Headers(response.headers);
        resHeaders.set("X-Omni-Node", baseUrl);
        return new Response(response.body, {
          status: response.status,
          headers: resHeaders
        });

      } catch (err) {
        // Network error or 2.5s timeout exceeded -> mark node dead and try next immediately
        deadNodes.set(baseUrl, Date.now());
      }
```

```
    }

    return new Response(
      JSON.stringify({ status: 503, error: "All federated free-tier nodes are temporarily busy." }),
      { status: 503, headers: { "Content-Type": "application/json" } }
    );
  },

  // 2. Cron Keep-Alive (Runs every 14 mins to prevent Render/Koyeb cold starts)
  async scheduled(event, env, ctx) {
    const warmTargets = [
      "https://core-api.koyeb.app/health",
      "https://core-api.onrender.com/health"
    ];
    await Promise.allSettled(warmTargets.map(url => fetch(url)));
  }
};
```

5.2 Oracle Cloud Primary Node Setup (`docker-compose.yml`)

On the primary Oracle Cloud Always Free ARM VM (2 OCPUs, 12 GB RAM), all application services run inside an isolated Docker bridge network behind a Cloudflare Tunnel (`cloudflared`) so that zero public ingress ports (`80/443`) are exposed on the host firewall:

```yaml
services:
  cloudflared:
    image: cloudflare/cloudflared:latest
    container_name: cf-tunnel
    restart: always
    command: tunnel --no-autoupdate run --token ${CF_TUNNEL_TOKEN}
    networks:
      - mesh-net

  core-api:
    build: .
    container_name: core-api
    restart: always
    environment:
      - PORT=3000
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=redis://redis:6379
      - JWT_SECRET=${JWT_SECRET}
    networks:
      - mesh-net

  redis:
    image: redis:7-alpine
    container_name: local-redis
    restart: always
    command: redis-server --maxmemory 2gb --maxmemory-policy allkeys-lru
    networks:
      - mesh-net

networks:
  mesh-net:
    driver: bridge
```

5.3 "Write Once, Deploy Everywhere" CI/CD Pipeline (`.github/workflows/deploy-mesh.yml`)

To operate the federated mesh like a unified cloud platform where a single `git push origin main` automatically deploys updates across every free provider simultaneously, connect a single GitHub repository to all downstream platforms:

- **Native GitHub Auto-Deploy:** Koyeb, Render, Vercel, Netlify, and Deno Deploy all watch the `main` branch directly via native GitHub integrations.

- **Automated Rollout Sequence on `git push origin main`:**
  1. **Vercel & Netlify** automatically build and deploy updated Edge and Serverless functions.
  2. **Koyeb & Render** automatically pull the latest commit, build the root `Dockerfile`, and perform a rolling container replacement.
  3. **GitHub Actions** connects to the Oracle Cloud ARM VM (or triggers Watchtower) to pull the latest Docker image and restart `core-api` with zero downtime.
  4. **Cloudflare Wrangler Action** publishes any routing updates to the Cloudflare Worker gateway (`worker.js`).

6. Security & Anti-Abuse Checklist

Operating a multi-cloud mesh across public PaaS endpoints requires strict origin authentication, write idempotency, and adherence to provider terms of service:

1. **Prevent Direct Origin Bypass (`X-Gateway-Secret`):**
   - *Threat:* Any external actor who discovers a raw backend URL (such as `core-api.onrender.com` or `core-api.koyeb.app`) could bypass Cloudflare WAF/rate limits and exhaust the provider's free quota directly.
   - *Mitigation:* The Edge Router injects a shared secret header (`X-Gateway-Secret: your-secret-key`) on every forwarded request, while Oracle ARM runs strictly behind a private Cloudflare Tunnel. Backend middleware immediately rejects any HTTP request missing a valid `X-Gateway-Secret` header with `403 Forbidden`.
2. **Enforce Write Idempotency on Mutating Requests (`Idempotency-Key`):**
   - *Threat:* If the primary Oracle node takes 2.6 seconds to process a `POST /api/payment` request, the Edge Router aborts at 2.5 seconds and retries the buffered payload on Koyeb—potentially executing the transaction twice.
   - *Mitigation:* Clients attach a unique `Idempotency-Key` UUID header to every mutating request. Backend middleware checks and locks this UUID in Redis/PostgreSQL before execution so failover retries return the cached result without creating duplicate records.
3. **Strict Compliance with Provider Terms of Service:**
   - *Policy:* Never register multiple fake accounts on a single provider (e.g., spinning up 10 separate Cloudflare or Render accounts) to evade quota limits.
   - *Architecture Alignment:* The Omni-Backend federation combines **1 legitimate account per distinct cloud provider** (1 Cloudflare + 1 Oracle + 1 Koyeb + 1 Render + 1 Vercel + 1 Netlify + 1 Deno + 1 Supabase/Neon). This multi-vendor architecture is 100% legal and fully compliant with every provider's Terms of Service.

7. Document Export & Local Rebuild Reference

When maintaining the Markdown source (`architecture.md`) alongside your repository, you can re-export or print updated versions locally using any of the following workflows:

- **In Chrome / Brave / Firefox (Fastest):** Open the rendered HTML or Markdown preview, press `Ctrl + P` → Set Destination to **"Save as PDF"** → Uncheck *"Headers and footers"* → Click **Save**.
- **On Arch Linux / Terminal (via Pandoc):** Save the Markdown source into `architecture.md` and run: `pandoc architecture.md -o Omni_Backend_Architecture.pdf --pdf-engine=wkhtmltopdf`
- **In VS Code:** Open `architecture.md` and run the **Markdown PDF** extension command (`Ctrl + Shift + P` → *Markdown PDF: Export (pdf)*).